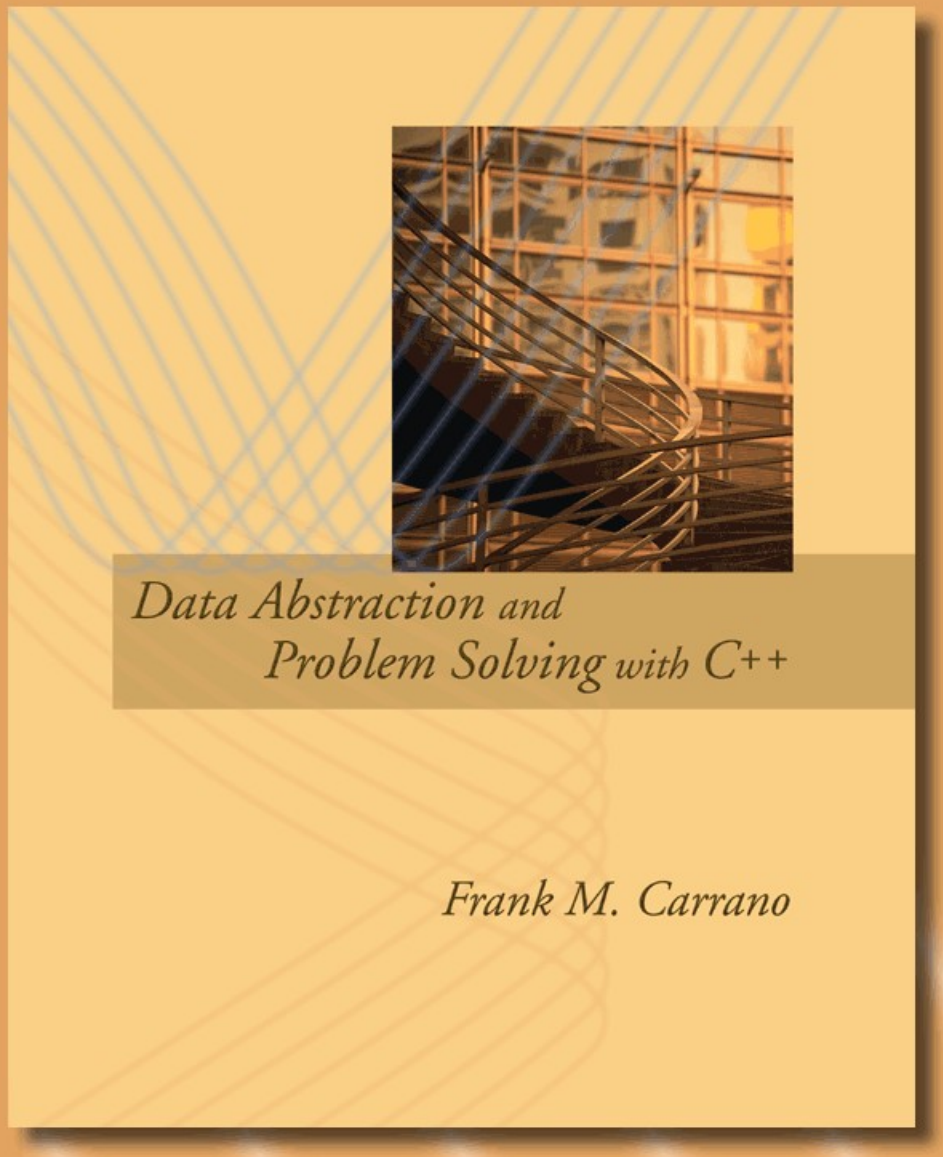
Chapter 4

Linked Lists

Slides edited by Dr. Selim Aksoy



PEARSON
Addison
Wesley

© 2005 Pearson Addison-Wesley. All rights reserved.

Preliminaries

- Options for implementing an ADT List
  - Array has a fixed size
    - Data must be shifted during insertions and deletions
  - Linked list is able to grow in size as needed
    - Does not require the shifting of items during insertions and deletions

© 2005 Pearson Addison-Wesley. All rights reserved

Preliminaries

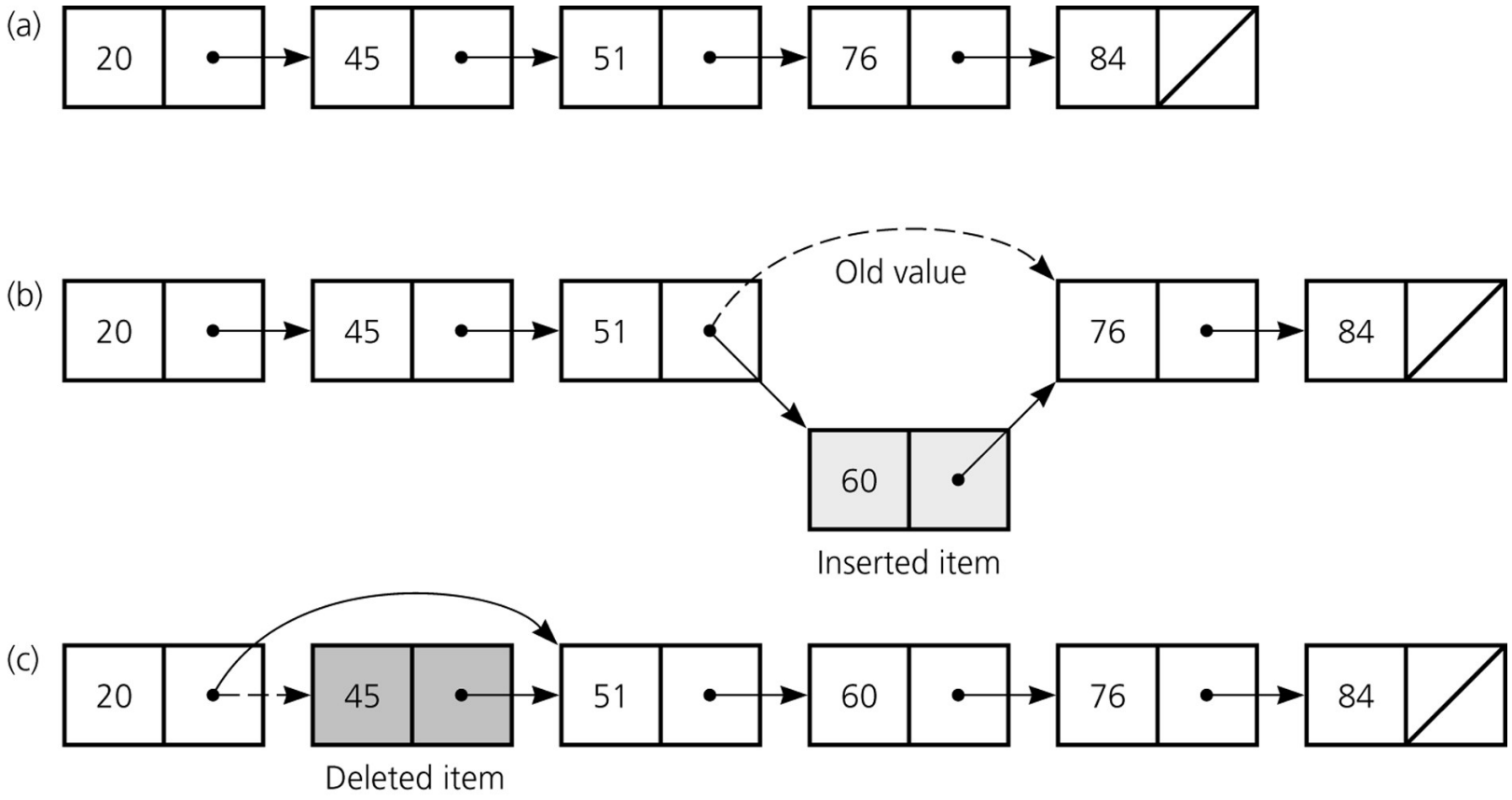


Figure 4.1 a) A linked list of integers; b) insertion; c) deletion

© 2005 Pearson Addison-Wesley. All rights reserved

Pointer-Based Linked Lists

- A node in a linked list is usually a `struct`

```
struct Node{
  int item;
  Node *next;
};
```

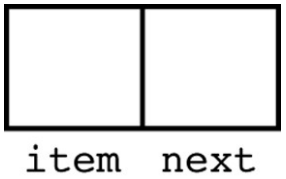


Figure 4.6 A node

- A node is dynamically allocated

```
Node *p;
p = new Node;
```

© 2005 Pearson Addison-Wesley. All rights reserved

Pointer-Based Linked Lists

- The head pointer points to the first node in a linked list
- If head is *NULL*, the linked list is empty

© 2005 Pearson Addison-Wesley. All rights reserved

Pointer-Based Linked Lists

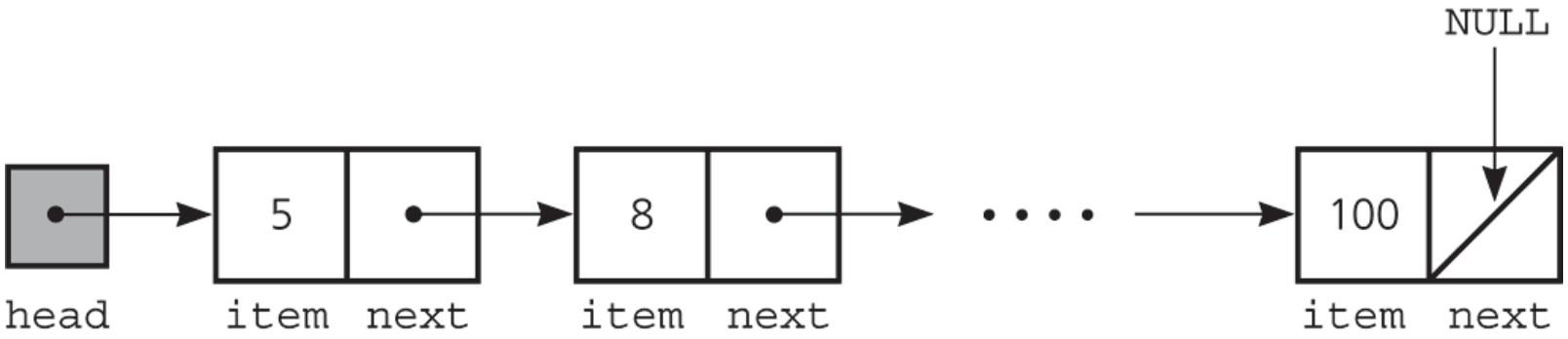


Figure 4.7 A head pointer to a list

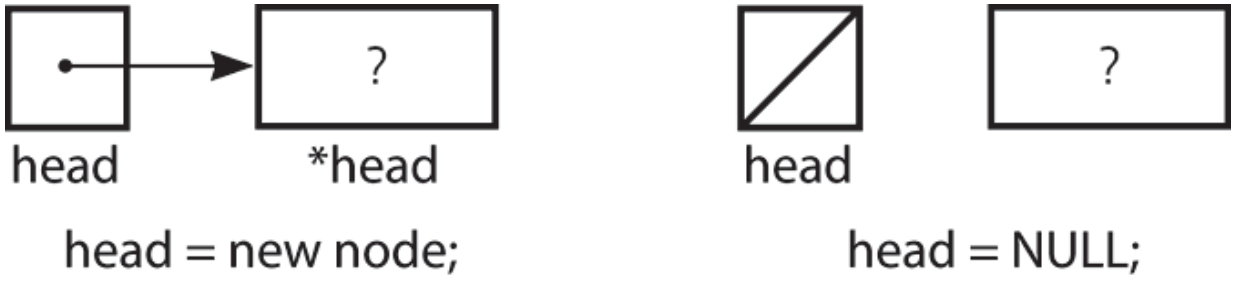


Figure 4.8 A lost cell

© 2005 Pearson Addison-Wesley. All rights reserved

Displaying the Contents of a Linked List

- Reference a node member with the -> operator

```
p->item;
```

- A traverse operation visits each node in the linked list
  - A pointer variable cur keeps track of the current node

```
for (Node *cur = head;
                                cur !=
NULL; cur = cur->next)
    cout << cur->item << endl;
```

© 2005 Pearson Addison-Wesley. All rights reserved

Displaying the Contents of a Linked List

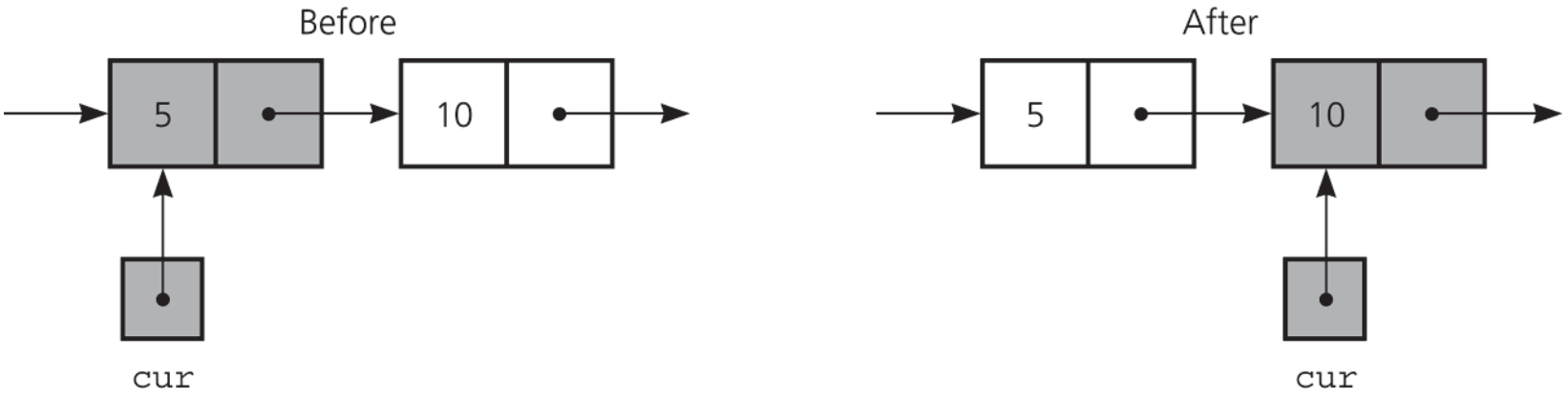


Figure 4.9

The effect of the assignment ***`cur = cur->next`***

© 2005 Pearson Addison-Wesley. All rights reserved

Deleting a Specified Node from a Linked List

- Deleting an interior node

```
prev->next=cur->next;
```

- Deleting the first node

```
head=head->next;
```

- Return deleted node to system

```
cur->next = NULL;
delete cur;
cur=NULL;
```

© 2005 Pearson Addison-Wesley. All rights reserved

Deleting a Specified Node from a Linked List

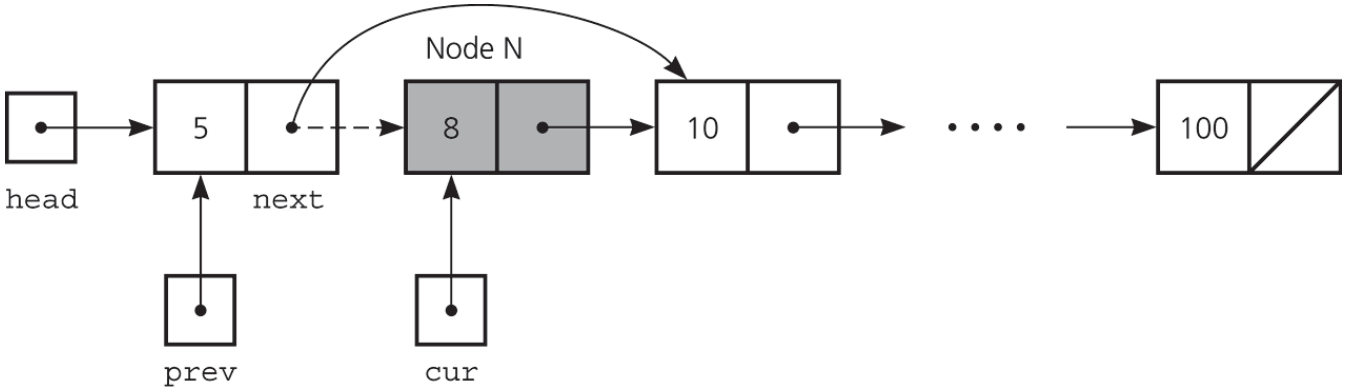


Figure 4.10 Deleting a node from a linked list

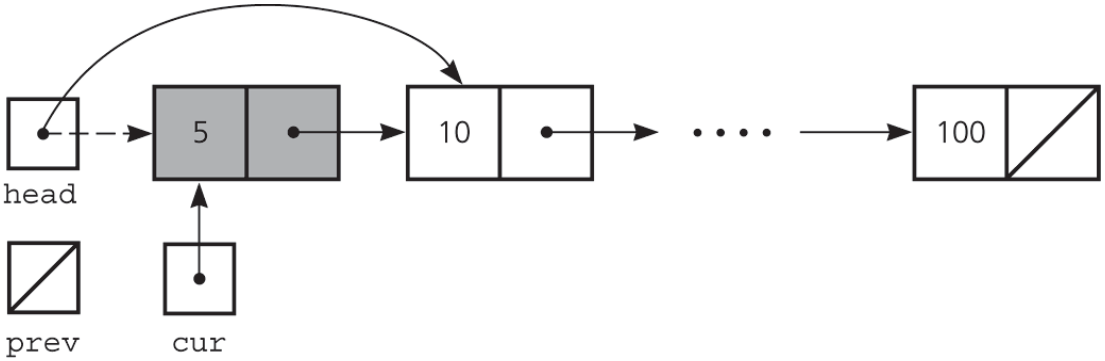


Figure 4.11 Deleting the first node

© 2005 Pearson Addison-Wesley. All rights reserved

Inserting a Node into a Specified Position of a Linked List

- To insert a node between two nodes

```
newPtr->next = cur;
prev->next = newPtr;
```

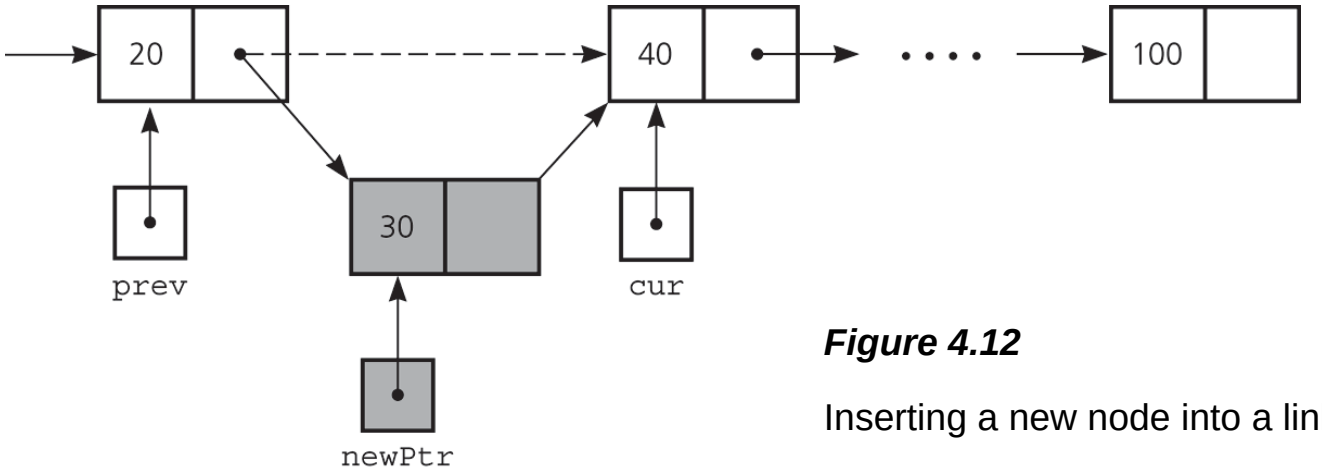


Figure 4.12

Inserting a new node into a linked list

© 2005 Pearson Addison-Wesley. All rights reserved

Inserting a Node into a Specified Position of a Linked List

- To insert a node at the beginning of a linked list

```
newPtr->next = head;
head = newPtr;
```

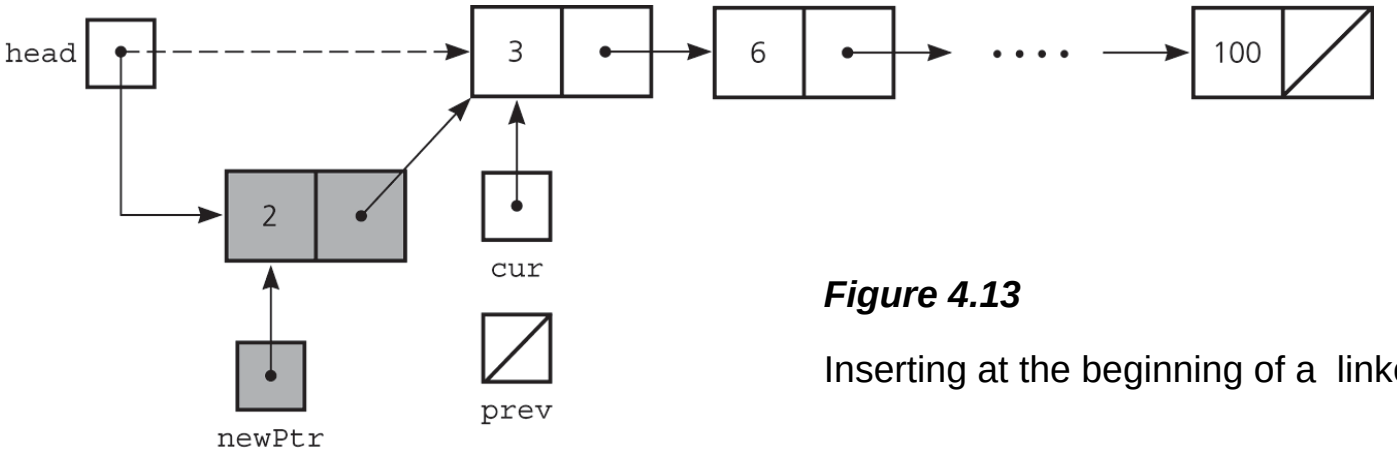


Figure 4.13

Inserting at the beginning of a linked list

© 2005 Pearson Addison-Wesley. All rights reserved

Inserting a Node into a Specified Position of a Linked List

- Inserting at the end of a linked list is not a special case if `cur` is *NULL*

```
newPtr->next = cur;
prev->next = newPtr;
```

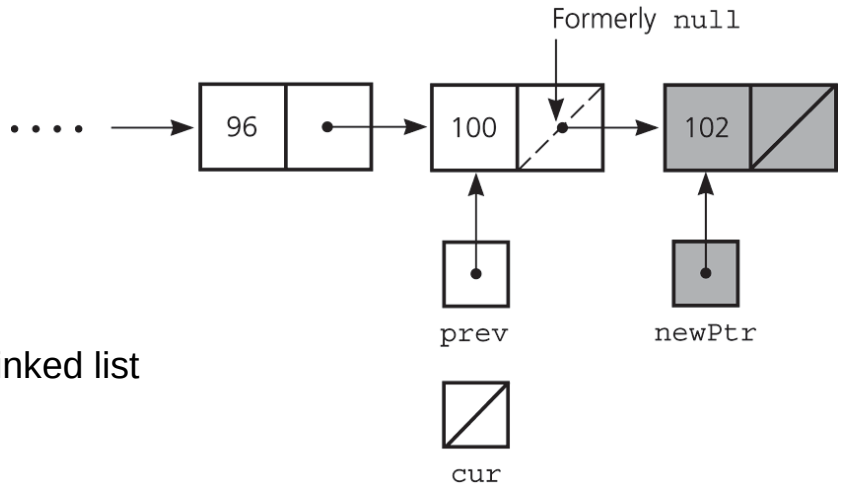


Figure 4.14

Inserting at the end of a linked list

© 2005 Pearson Addison-Wesley. All rights reserved

Inserting a Node into a Specified Position of a Linked List

- Determining the point of insertion or deletion for a sorted linked list of objects

```
for(prev = NULL, cur= head;
        (cur != null)&&
    (newValue > cur->item);
        prev = cur, cur = cur->next);
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

- Public methods
  - `isEmpty`
  - `getLength`
  - `insert`
  - `remove`
  - `retrieve`
- Private method
  - `find`
- Private Data Members
  - `head`
  - `size`
- Local variables to member functions
  - `cur`
  - `prev`

© 2005 Pearson Addison-Wesley. All rights reserved

Constructors and Destructors

- Default constructor initializes size and head
- Copy constructor allows a deep copy
  - Copies the array of list items and the number of items
- A destructor is required for dynamically allocated memory

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
// ****************************************
// Header file ListP.h for the ADT list.
// Pointer-based implementation.
//****************************************

typedef desired-type-of-list-item ListItemType;

class List{

// constructors and destructor:
public:
   List();                          // default constructor
   List(const List& aList); // copy constructor
   ~List();                         // destructor
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
// list operations:
   bool isEmpty() const;

   int getLength() const;

   bool insert(int index, ListItemType newItem);

   bool remove(int index);

   bool retrieve(int index,
                 ListItemType& dataItem) const;
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
private:

   struct ListNode{          // a node on the list
      ListItemType item; // a data item on the list
      ListNode *next;    // pointer to next node
   };


   int         size;  // number of items in list
   ListNode *head;  // pointer to linked list
                       // of items


   ListNode *find(int index) const;
   // Returns a pointer to the index-th node
   // in the linked list.
};
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
// ****************************************************
// Implementation file ListP.cpp for the ADT list
// Pointer-based implementation.
// ****************************************************

#include "ListP.h"      // header file
#include <cstddef>      // for NULL

List::List(): size(0), head(NULL){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
List::List(const List& aList): size(aList.size){
   if (aList.head == NULL)
      head = NULL;  // original list is empty
   else {
      // copy first node
      head = new ListNode;
      head->item = aList.head->item;
      // copy rest of list
      ListNode *newPtr = head;  // new list ptr

      for (ListNode *origPtr = aList.head->next;
                     origPtr != NULL;
                     origPtr = origPtr->next){
         newPtr->next = new ListNode;
         newPtr = newPtr->next;
         newPtr->item = origPtr->item;
      }
      newPtr->next = NULL;
   }
}  // end copy constructor
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
List::~List(){

   while (!isEmpty())
      remove(1);

} // end destructor
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
bool List::isEmpty() const{
   return size == 0;
}  // end isEmpty
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
int List::getLength() const{
   return size;
}  // end getLength
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
List::ListNode *List::find(int index) const{
// Locates a specified node in a linked list.
// Precondition: index is the number of the
// desired node.
// Postcondition: Returns a pointer to the
// desired node. If index < 1 or index > the
// number of nodes in the list, returns NULL.

   if ( (index < 1) || (index > getLength()) )
      return NULL;

   else{ // count from the beginning of the list
      ListNode *cur = head;
      for (int skip = 1; skip < index; ++skip)
         cur = cur->next;
      return cur;
   }
}  // end find
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
bool List::retrieve(int index,
                    ListItemType& dataItem) const {

   if ((index < 1) || (index > getLength()))
      return false;

   // get pointer to node, then data in node
   ListNode *cur = find(index);
   dataItem = cur->item;

   return true;

} // end retrieve
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
bool List::insert(int index, ListItemType newItem) {

   int newLength = getLength() + 1;

   if ((index < 1) || (index > newLength))
      return false;

   ListNode *newPtr = new ListNode;
   size = newLength;
   newPtr->item = newItem;

   if (index == 1){
      newPtr->next = head;
      head = newPtr;
   }
   else{
      ListNode *prev = find(index-1);
      newPtr->next = prev->next;
      prev->next = newPtr;
   }
   return true;

} // end insert
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation of the ADT List

```
bool List::remove(int index) {

   ListNode *cur;

   if ((index < 1) || (index > getLength()))
      return false;

   --size;
   if (index == 1){
      cur = head;
      head = head->next;
   }
   else{
      ListNode *prev = find(index-1);
      cur = prev->next;
      prev->next = cur->next;
   }
   cur->next = NULL;
   delete cur;
   cur = NULL;

   return true;

}  // end remove
```

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Array-Based and Pointer-Based Implementations

- Size
  - Increasing the size of a resizable array can waste storage and time
- Storage requirements
  - Array-based implementations require less memory than a pointer-based ones

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Array-Based and Pointer-Based Implementations

- Access time
  - Array-based: constant access time
  - Pointer-based: the time to access the ith node depends on i
- Insertion and deletions
  - Array-based: require shifting of data
  - Pointer-based: require a list traversal

© 2005 Pearson Addison-Wesley. All rights reserved

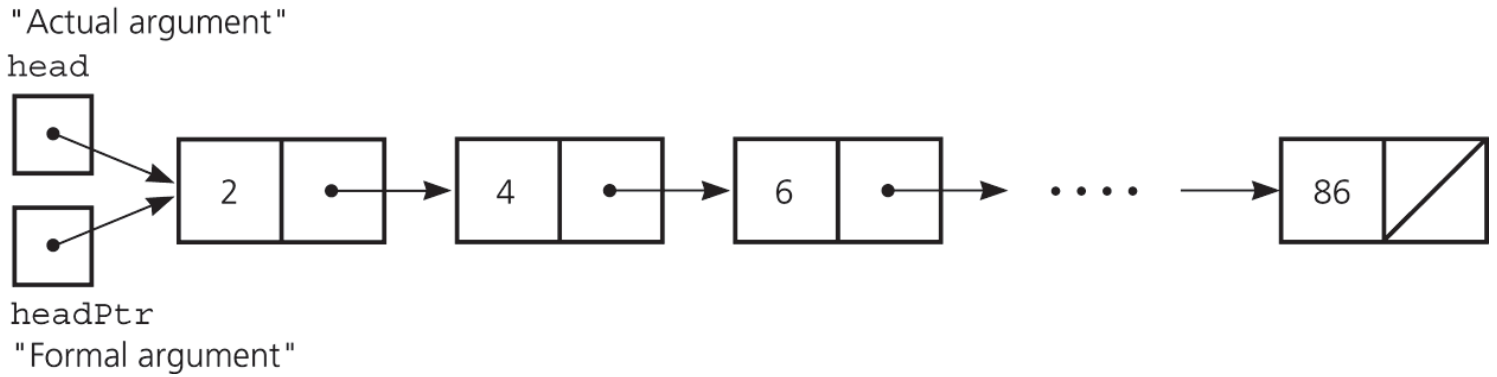
Saving and Restoring a Linked List by Using a File

- Use an external file to preserve the list between runs
- Do not write pointers to a file, only data
- Recreate the list from the file by placing each item at the end of the list
  - Use a tail pointer to facilitate adding nodes to the end of the list
  - Treat the first insertion as a special case by setting the tail to head

© 2005 Pearson Addison-Wesley. All rights reserved

Passing a Linked List to a Function

- A function with access to a linked list's `head` pointer has access to the entire list



© 2005 Pearson Addison-Wesley. All rights reserved

Objects as Linked List Data

- Data in a linked list node can be an instance of a class

```
typedef ClassName ItemType;
struct Node{
   ItemType item;
    Node *next;
};
Node *head;
```

© 2005 Pearson Addison-Wesley. All rights reserved

Circular Linked Lists

- Last node references the first node
- Every node has a successor
- No node in a circular linked list contains *NULL*

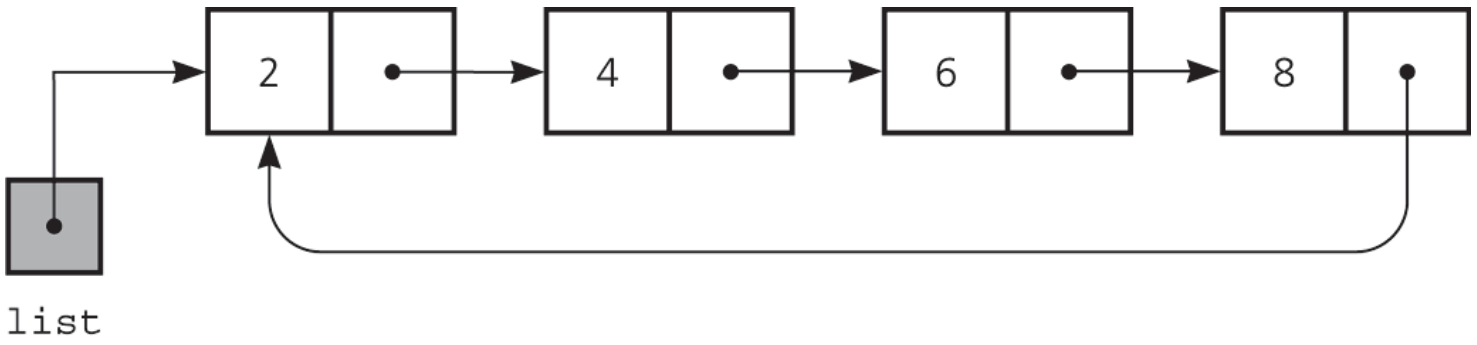


Figure 4.25 A circular linked list

© 2005 Pearson Addison-Wesley. All rights reserved

Circular Linked Lists

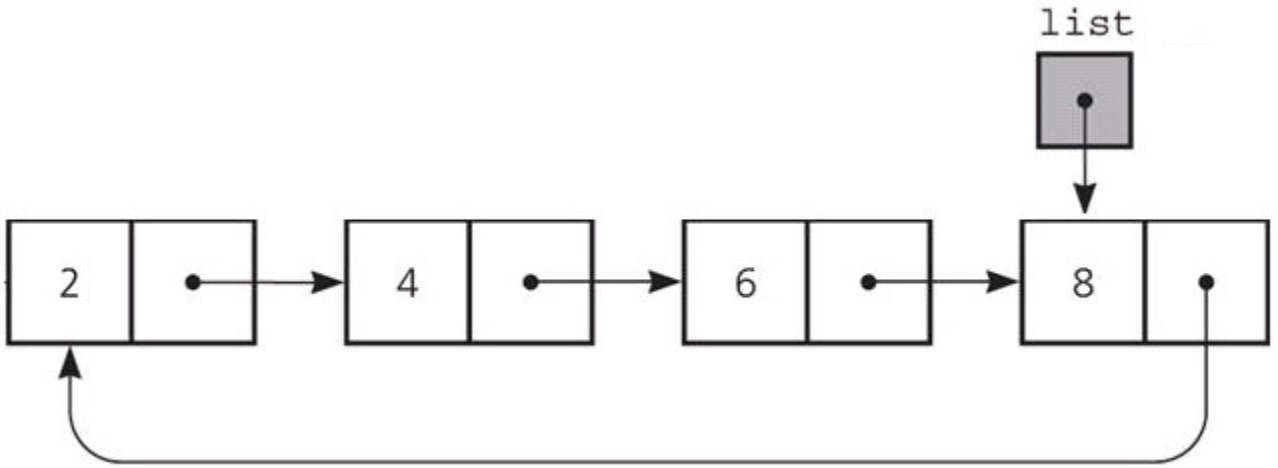


Figure 4.26 A circular linked list with an external pointer to the last node

© 2005 Pearson Addison-Wesley. All rights reserved

Circular Linked Lists

```
// display the data in a circular linked list;
// list points to its last node

if (list != NULL){
   // list is not empty
   Node *first = list->next; // point to first node

   Node *cur = first;        // start at first node

   // Loop invariant: cur points to next node to
   // display
   do{
      display(cur->item);    // write data portion
      cur = cur->next;       // point to next node
   } while (cur != first);   // list traversed?

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

Dummy Head Nodes

- Dummy head node
  - Always present, even when the linked list is empty
  - Insertion and deletion algorithms initialize `prev` to reference the dummy head node, rather than *NULL*

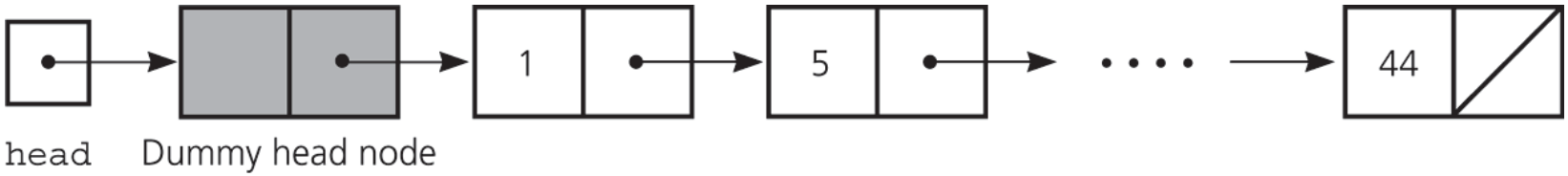


Figure 4.27 A dummy head node

© 2005 Pearson Addison-Wesley. All rights reserved

Doubly Linked Lists

- Each node points to both its predecessor and its successor
- Circular doubly linked list
  - `precede` pointer of the dummy head node points to the last node
  - `next` reference of the last node points to the dummy head node
  - No special cases for insertions and deletions

© 2005 Pearson Addison-Wesley. All rights reserved

Doubly Linked Lists

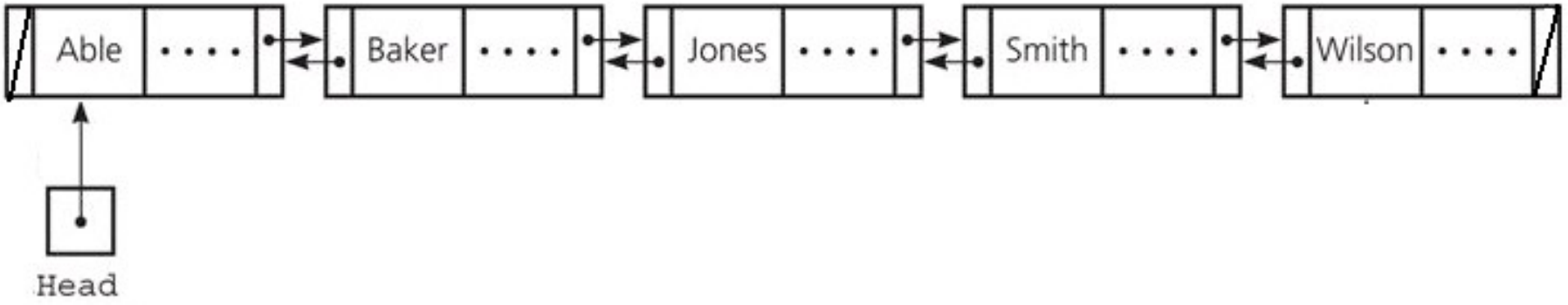


Figure 4.28 A doubly linked list

© 2005 Pearson Addison-Wesley. All rights reserved

Doubly Linked Lists

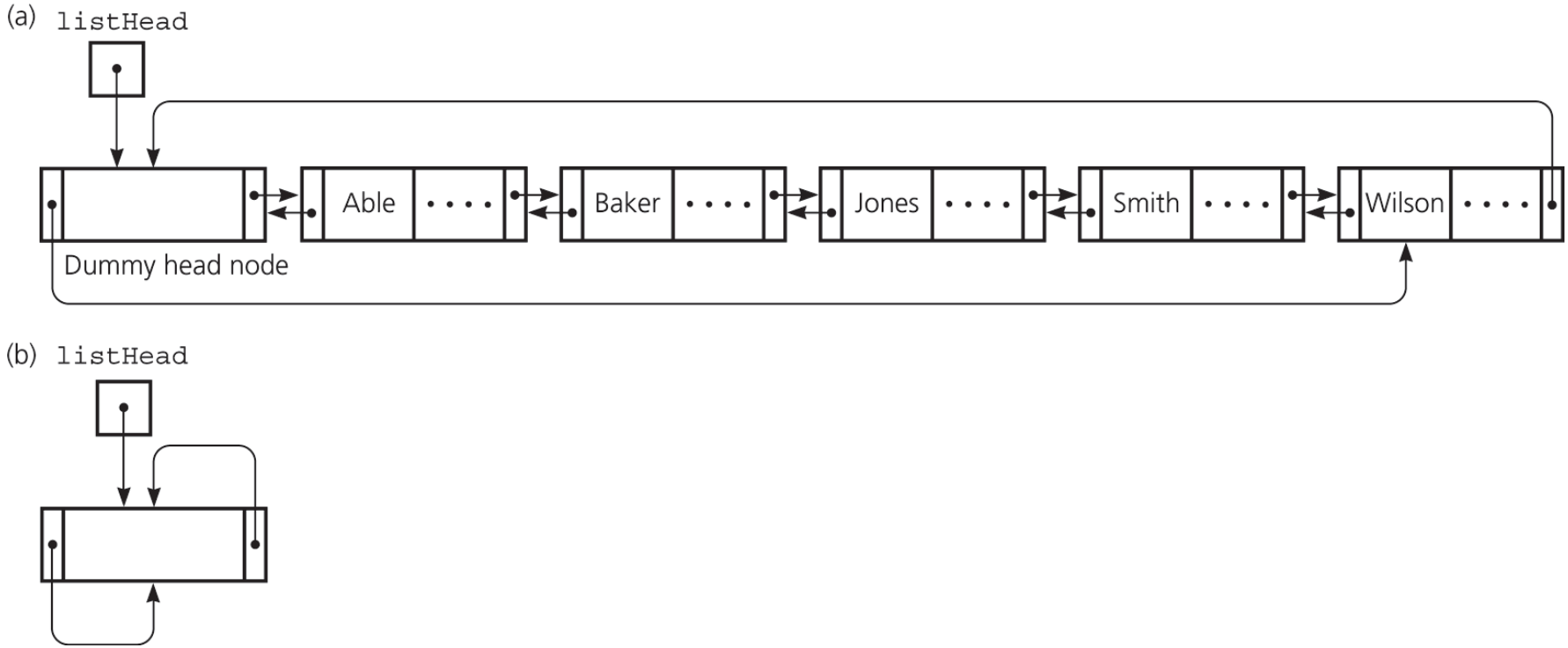


Figure 4.29 (a) A circular doubly linked list with a dummy head node

(b) An empty list with a dummy head node

© 2005 Pearson Addison-Wesley. All rights reserved

Doubly Linked Lists

- To delete the node to which `cur` points

```
(cur->precede)->next = cur->next;
(cur->next)->precede = cur->precede;
```

- To insert a new node pointed to by `newPtr` before the node pointed to by `cur`

```
newPtr->next = cur;
newPtr->precede = cur->precede;
cur->precede = newPtr;
newPtr->precede->next = newPtr;
```

© 2005 Pearson Addison-Wesley. All rights reserved

Summary

- Each pointer in a linked list is a pointer to the next node in the list
- Algorithms for insertions and deletions in a linked list involve traversing the list and performing pointer changes
  - Inserting a node at the beginning of a list and deleting the first node of a list are special cases

© 2005 Pearson Addison-Wesley. All rights reserved

Summary

- In a circular linked list, the last node points to the first node
- Dummy head nodes eliminate the special cases for insertion into and deletion from the beginning of a linked list

© 2005 Pearson Addison-Wesley. All rights reserved